

Backend Internship Task

1. Objective & Scope

The objective of this assessment is to evaluate your fundamental backend focused web development and API integration capabilities. You are required to build a lightweight, production-ready Dynamic Blog Engine utilizing **Next.js (App Router)**, **Prisma ORM**. (you have permission to use nest, express, node)

This test measures your capability to architect a relational database schema, design relational API endpoints with automated slugification, enforce data integrity, and build a highly performant user interface utilizing utility-first CSS and modern dynamic routing patterns.

2. Core Technical Stack

Framework: Next.js 14+ / 15 - preferable or - nest, express, node - any

Database ORM: Prisma Client paired with an isolated SQLite database instance (you can use any database via Prisma ORM)

Language: Javascript/Typescript

3. Database Architecture & Schema

Initialize a local Prisma environment configuring an isolated SQLite binary data layer. Your schema layout must reflect a standalone Post model engineered to support structured document entries and optimized indexed URL lookups.

```
model Post {
  id String @id @default(uuid())
  title String
  slug String @unique
  image String // To store target structural image URL path
  description String // Markdown or clean body text block
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}
```

Schema Implementation Note:

Ensure that your migrations are executed locally running (***npx prisma migrate dev --name init***) to synchronize the physical local database state before proceeding with backend API construction.

4. Backend API Specifications

You must construct standard Route Handlers to perform administrative and consumption operations securely.

A. Resource Creation Endpoint: POST /api/posts

This endpoint handles the consumption of payload variables sent from your client administrative creation form.

Payload Requirements: JSON body parsing absolute inputs for title , image , and description .

Automated Slugification: The backend must deterministically extract the title property and safely translate it into an optimized slug string (**alphanumeric characters and hyphens only, completely lowercase, devoid of consecutive hyphens**).

Collision Management: Ensure clean record generation via Prisma. If a slug collision occurs, append a unique hash or throw an elegant validation response status code.

```
// Example helper logic for slug transformation

const generateSlug = (title: string): string => {
  return title
    .toLowerCase()
    .trim()
    .replace(/[^\w-]/g, '') // Remove special characters
    .replace(/\s+/g, '-') // Convert spaces to single hyphens
    .replace(/-+/g, '-'); // Deduplicate sequential hyphens
};
```

B. Dynamic Read Optimization Endpoint: GET /api/posts/[slug]

Responsible for handling immediate runtime fetching strategies when building server-side or hydrate-ready page layers.

Parameter Lookup: Identify dynamic slug components cleanly within the route contextual arguments.

Database Validation: Utilize `prisma.post.findUnique()` queries hitting the indexed slug field.

Response Codes: Return a clean 200 OK with data objects on successful lookups, or output standard structural 404 Not Found JSON blocks if records don't exist.

5. Frontend Core Pages & Interface Design

The interface layer must build an interactive front-end with clean typography and structural spacing.

A. Administrative Form (`/blog/create`)

Construct a dedicated portal view layout featuring a clean, responsive card element containing:

- A text input for the Post Title.
- A text input mapping the Image URL string.
- A multi-line Description input.
- Robust UI submission buttons featuring state handling (loading/disabled states).

B. Dynamic Reader Layout (`/blog/[slug]`)

A dynamic user interface rendered cleanly from backend slug parameters:

- **Layout Constraints:** Constrained center-column container layout (e.g., `max-w-3xl mx-auto`).
 - **Typography Hierarchy:** Prominent text sizes for headers paired with light sub-elements.
 - **Visual Components:** Fully rounded modern structural card shadow design details on image Structures.
-

6. Submission Protocol Instructions:

Please structure your project code cleanly inside a single GitHub repository. Include a structured [README.md](#) detailing clear documentation command steps detailing environment setups, step-by-step local initializations, and any core structural implementation assumptions.
